

Tecnologia em Jogos Digitais • Texturização

Quanto pesa uma textura?

UDIMs, compressão, mipmaps e channel packing

Semana 15 — Otimizar não é só ter menos texturas

IFMS • Semana 15

Objetivos de hoje

Ao final da semana você será capaz de:

- Explicar o que são **UDIMs** e quando um asset justifica seu uso
- Diferenciar formatos de compressão **BC1, BC3, BC7** por tipo de mapa
- Entender **mipmaps** e seu impacto na escolha de resolução
- Executar **channel packing** montando um mapa **ORM**
- **Justificar** a resolução de cada asset por importância visual
- **Quantificar** a economia de memória em uma tabela antes/depois



**Estas duas imagens parecem iguais na tela.
Qual pesa mais em memória — e por
quanto?**

De onde viemos: menos texturas, mas ainda pesadas

Nas Semanas 13 e 14, otimizamos a **quantidade** de texturas.

- **Atlas** — agrupou objetos fixos em um único espaço 0–1
- **Trim Sheet** — reutilizou uma faixa por tiling
- Resultado: **menos arquivos** distintos

DICA

Um asset pode já ter o número mínimo de texturas e ainda **desperdiçar memória**: se cada arquivo for maior que o preciso, sem mipmaps ou com mapas grayscale soltos.

UDIMs: quando um tile 0-1 não basta

O UV pode ocupar **múltiplos tiles numerados** (1001, 1002, 1003...), cada um um espaço 0-1 independente, com sua própria textura de alta resolução.

- Usado quando **um único asset** precisa de mais detalhe do que um tile comporta
- Tipicamente o objeto **hero** do kit — grande e próximo da câmera
- Cada tile carrega uma textura própria, sem borrar o detalhe

DICA

O espaço UV 0-1 é uma **folha de papel**. Até agora tudo coube em uma folha — mesmo vários objetos no atlas. UDIM é usar **mais de uma folha para o mesmo objeto**.

UDIM é exceção, não regra

A maioria dos assets de um kit modular **não precisa** de UDIM.

- Atlas e trim já resolvem a maior parte dos casos (S13–14)
- UDIM se justifica só para o asset **mais importante e mais próximo** da câmera
- Se houver um objeto hero com muito mais detalhe que os demais

⚠ ATENÇÃO

Empolgação com o recurso novo não é justificativa. A pergunta é: **esse asset perde detalhe perceptível em um único tile?**

Compressão: BC1, BC3, BC7

Motores não guardam texturas como PNG — usam formatos comprimidos para GPU, decodificados em tempo real.

- **BC1** — compressão agressiva, sem alpha. Bom para Albedo sem transparência e Metallic
- **BC3** — inclui canal alpha completo. Para transparência real (folhagem, grades)
- **BC7** — alta qualidade, arquivo maior. Ideal para **Normal Maps** e Albedo hero

DICA

A pergunta não é "qual formato é melhor", é "**qual erro esse mapa específico pode tolerar sem que o olho perceba**".

Mipmaps: por que textura grande demais não é "mais seguro"

Mipmaps são versões **pré-calculadas** da textura em resoluções decrescentes, trocadas conforme a distância à câmera.

- Geradas **automaticamente** pelo motor
- Evitam **aliasing** em superfícies distantes e economizam banda
- O motor já troca por uma versão menor quando o objeto está longe

✓ BOAS PRÁTICAS

O detalhe extra de uma textura maior que o necessário **nunca é visto** de perto o suficiente para justificar o peso. Escolha bem o tamanho original.

Channel packing: o mapa ORM

Roughness, Metallic e AO são mapas em **escala de cinza** — cada um usa só 1 canal, mas ocupa um arquivo RGB inteiro se exportado sozinho.

Antes

3 arquivos

Roughness · Metallic · AO

Cada um grayscale, arquivo próprio.

Depois (ORM)

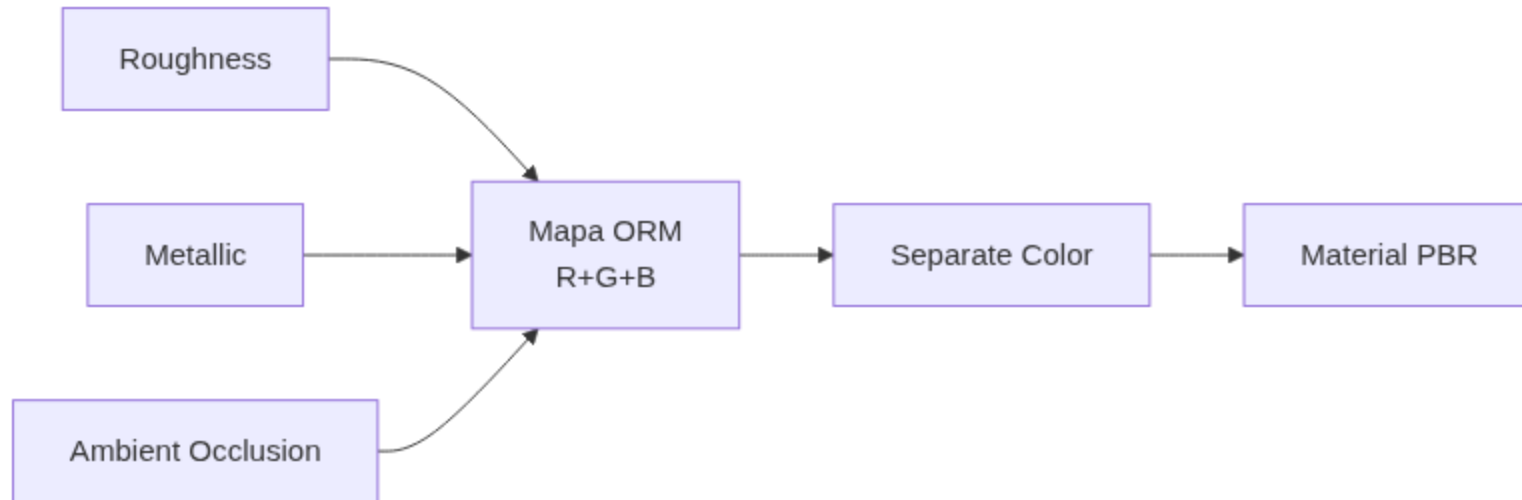
1 arquivo

R = Roughness

G = Metallic

B = AO

O pipeline do channel packing



Três mapas grayscale entram nos canais R, G e B; no material, um nó **Separate Color** os devolve a cada input.

Só grayscale entra no ORM

Channel packing **só funciona** porque os três mapas de origem já eram escala de cinza.

⊘ ERRO COMUM

Incluir Albedo no ORM — o Albedo é cor real (RGB), não um valor único. Ele não cabe no esquema de canais e quebra a técnica.

A pergunta-chave: *esse mapa usa cor real, ou é uma escala de cinza representando um valor único?*

Resolução: por critério, não por hábito

Nem todo asset merece a mesma resolução. Classifique **antes** de decidir.

- **Hero** — grande, próximo da câmera, foco da cena → 2048
- **Secundário** — presença regular, média distância → 1024
- **Fundo** — pequeno, repetido, distante → 512

⊘ ERRO COMUM

"Deixei tudo em 2048 porque já estava assim" — resolução por conveniência, não por importância visual em cena.

Erros comuns

⊘ ERRO COMUM

Inversão de canais — Metallic no R em vez do G. Material fisicamente incoerente. Verifique cada canal isolado contra o mapa de origem.

⊘ ERRO COMUM

Albedo no channel packing — cor real não é grayscale. Não entra no ORM.

⊘ ERRO COMUM

Compressão genérica — mesmo formato para todos os mapas. Normal Map tolera muito menos erro que Albedo.

Na indústria

Nenhum jogo entrega texturas soltas em PNG cru. **ORM, compressão por tipo de mapa e mipmaps** são rotina de produção — não um extra.

O que separa o portfólio de estudante do portfólio de quem entende produção é saber **justificar com números** por que cada textura pesa o que pesa.



Resumo

- **UDIM** — múltiplos tiles para um asset hero; exceção, não regra
- **Compressão** — BC1/BC3/BC7 conforme o erro que cada mapa tolera
- **Mipmaps** — o motor troca por versões menores; não superdimensiona
- **Channel packing (ORM)** — R/G/B = Roughness/Metallic/AO, só grayscale
- **Resolução** por importância visual; **tabela antes/depois** como evidência

Agora: demonstração

A seguir, ao vivo: montar um **mapa ORM** combinando Roughness, Metallic e AO, reconfigurar o material no Blender e comparar o peso antes/depois.

A pergunta que você leva ao estúdio: **quais mapas do meu kit já são grayscale e podem virar um único ORM?**

